

ARCHITECTURE DECISION RECORD (ADR) — SIMT MTs

Proyek: Sistem Informasi Manajemen Sekolah Madrasah Terpadu (SIMT MTs)

Versi Dokumen: 1.0 — FINAL

Tanggal: 4 Maret 2026

Penulis: Founder / Tech Lead

Status Dokumen: BINDING — Keputusan di bawah ini bersifat FINAL dan mengikat seluruh pengembangan MVP.

Revisi hanya dapat dilakukan setelah MVP ter-launch dan terbukti memerlukan perubahan arsitektural.

Daftar Isi

- [ADR-001: Frontend Stack — Hybrid Blade + Next.js](#adr-001-frontend-stack--hybrid-blade--nextjs)
- [ADR-002: Primary Key Strategy — BIGSERIAL untuk MVP](#adr-002-primary-key-strategy--bigserial-untuk-mvp)
- [ADR-003: RBAC Implementation — Spatie Laravel Permission with Teams](#adr-003-rbac-implementation--spatie-laravel-permission-with-teams)
- [ADR-004: Budget & Scope — Lean MVP Rp 5 Juta](#adr-004-budget--scope--lean-mvp-rp-5-juta)
- [ADR-005: Database Engine — MySQL 8 untuk MVP](#adr-005-database-engine--mysql-8-untuk-mvp)
- [ADR-006: Caching & Queue — Redis Multi-Purpose](#adr-006-caching--queue--redis-multi-purpose)
- [ADR-007: Tenant Isolation — Single-Database with Global Scope](#adr-007-tenant-isolation--single-database-with-global-scope)
- [Appendix: Document Update Checklist](#appendix-document-update-checklist)

Pendahuluan

Dokumen ini merupakan Architecture Decision Record (ADR) resmi untuk proyek SIMT MTs (Sistem Informasi Manajemen Sekolah Madrasah Terpadu). ADR ini menyelesaikan **7 inkonsistensi kritis** yang ditemukan di antara 49 dokumen proyek yang telah diproduksi selama fase perencanaan. Setiap keputusan di bawah ini telah ditetapkan sebagai keputusan FINAL yang mengikat seluruh pengembangan MVP. Inkonsistensi yang ditemukan mencakup konflik antara pilihan teknologi, anggaran, strategi database, implementasi RBAC, mekanisme caching, strategi primary key, dan model isolasi tenant. Keputusan-keputusan ini dibuat dengan mempertimbangkan kendala nyata: VPS dengan RAM 2GB, anggaran Rp 5 juta untuk 3 bulan MVP, satu founder dengan bantuan AI, dan target pengguna Madrasah Tsanawiyah di Indonesia yang membutuhkan sistem yang ringan, cepat, dan terjangkau.

ADR-001: Frontend Stack — Hybrid Blade + Next.js

Konteks

Terdapat konflik signifikan antara dua kelompok dokumen mengenai pilihan frontend stack. **Dokumen 05, 09, 17-19** mereferensikan **Vue.js 3 + Vite** sebagai solusi frontend untuk antarmuka admin, dengan asumsi arsitektur Single Page Application (SPA) yang modern. Sementara itu, **Dokumen 00, 24-26** mereferensikan arsitektur **hybrid Blade (Admin) + Next.js (Portal Ortu)**, di mana panel admin menggunakan Blade templating engine bawaan Laravel dan portal orang tua menggunakan Next.js sebagai aplikasi terpisah yang mobile-first dan PWA-capable. Konflik ini berdampak langsung pada arsitektur autentikasi, strategi deployment, penggunaan memori VPS, dan kompleksitas pengembangan.

Keputusan

Blade + Next.js hybrid sebagaimana didefinisikan dalam Dokumen 24-26. Vue.js 3 + Vite **DITOLAK** untuk fase MVP maupun jangka panjang. Panel admin menggunakan Laravel Blade dengan Alpine.js untuk interaktivitas ringan. Portal orang tua menggunakan Next.js dengan App Router untuk pengalaman mobile-first.

Rasional

Keputusan ini didasarkan pada empat pertimbangan teknis dan operasional yang kritis. **Pertama**, Blade SSR (Server-Side Rendering) memberikan Time to First Byte (TTFB) yang jauh lebih rendah dibandingkan SPA pada VPS 2GB RAM. Dengan Blade, server merender HTML lengkap dan mengirimkannya langsung ke browser tanpa memerlukan round-trip tambahan untuk fetch JavaScript bundle dan data API. Pada VPS yang terbatas, setiap megabyte RAM dan setiap milisecond response time sangat berharga. **Kedua**, Blade menghilangkan kebutuhan untuk membangun dan memelihara API authentication terpisah untuk panel admin. Dengan Blade, session-based authentication bawaan Laravel sudah cukup — tidak perlu setup Sanctum/Passport API tokens, tidak perlu menangani CORS, dan tidak perlu mengelola state sinkronisasi antara frontend dan backend. **Ketiga**, Alpine.js menyediakan reaktivitas yang cukup untuk kebutuhan interaktivitas panel admin — dropdown, modal, tab switching, form validation feedback, dan toggle semuanya dapat ditangani oleh Alpine.js tanpa overhead framework SPA penuh. Panel admin sekolah bukan aplikasi real-time yang memerlukan WebSocket atau state management kompleks seperti Vuex/Pinia. **Keempat**, Next.js secara eksklusif digunakan hanya untuk portal orang tua karena use case yang berbeda secara fundamental — orang tua mengakses dari perangkat mobile, membutuhkan PWA untuk offline capability, dan mengharapkan pengalaman seperti aplikasi native. Next.js App Router dengan SSR/SSG memberikan performa mobile yang optimal sekaligus memungkinkan fitur PWA. Dengan membatasi Next.js hanya untuk satu sub-aplikasi, kita menjaga kompleksitas tetap terkendali dan penggunaan resource VPS tetap efisien.

Konsekuensi

- **Tidak ada SPA untuk admin panel.** Seluruh navigasi admin menggunakan full-page reload tradisional. Ini berarti tidak ada client-side routing, tidak ada seamless page transitions, dan tidak ada optimistic UI updates. Namun, untuk penggunaan admin sekolah yang bersifat data-entry intensif, full-page reload dengan Blade SSR justru memberikan pengalaman yang lebih konsisten dan dapat diandalkan dibandingkan SPA yang rentan terhadap state inconsistency.
- **Harus memelihara sistem template ganda.** Tim (atau founder) harus menguasai dua ekosistem templating: Blade untuk admin dan React/Next.js untuk portal orang tua. Ini menambah beban kognitif dan memerlukan dokumentasi yang jelas untuk boundary antara keduanya. Aturan yang berlaku: semua fitur manajemen sekolah menggunakan Blade, semua fitur akses orang tua menggunakan Next.js. Tidak ada overlap.
- **Alpine.js menggantikan Vue reactivity.** Developer harus berpindah paradigma dari Vue's reactive data system ke Alpine.js yang lebih declarative dan lebih sederhana. Alpine.js tidak memiliki computed properties, watchers, atau component lifecycle hooks yang sama dengan Vue. Semua logic yang memerlukan state kompleks harus tetap berada di backend Laravel, bukan di frontend.
- **Deployment pipeline terpisah.** Blade ter-deploy sebagai bagian dari Laravel application, sedangkan Next.js memerlukan build step terpisah dan dapat di-deploy sebagai standalone Node.js process atau static export. Ini mempengaruhi konfigurasi Nginx reverse proxy dan CI/CD pipeline.

Catatan Implementasi

- Alpine.js diinstall via CDN atau npm (alpinejs + @alpinejs/focus + @alpinejs/persist)
- Next.js portal berjalan di port terpisah (default :3000) di-belakang Nginx reverse proxy
- Tidak ada shared component library antara Blade dan Next.js — masing-masing memiliki design system sendiri

- Blade menggunakan Tailwind CSS dengan komponen Blade anonymous components (resources/views/components/)
- Next.js menggunakan Tailwind CSS dengan shadcn/ui components

Status: FINAL

ADR-002: Primary Key Strategy — BIGSERIAL untuk MVP

Konteks

Terdapat inkonsistensi fundamental dalam strategi primary key antar dokumen. **Dokumen 06** menggunakan **UUID (gen_random_uuid)** sebagai primary key untuk semua tabel, mengikuti pendekatan yang umum dalam arsitektur microservices dan distributed systems. Sementara itu, **Dokumen 09 dan 39** menggunakan **BIGSERIAL** (auto-increment 8-byte integer) sebagai primary key, mengikuti konvensi Laravel default dan pendekatan tradisional monolithic database. Pilihan ini memiliki dampak signifikan pada ukuran database, performa index, kompatibilitas framework, dan strategi scaling di masa depan.

Keputusan

BIGSERIAL untuk semua tabel MVP. UUID dicadangkan untuk Phase 2 ketika sistem perlu mendukung multi-region deployment. Pengecualian: `tenant_id` boleh menggunakan UUID untuk identifikasi tenant lintas database, tetapi `tenant_id` bukan primary key tabel — ia adalah foreign key yang merujuk ke tabel `tenants` yang primary key-nya tetap BIGSERIAL.

Rasional

Keputusan ini didasarkan pada analisis trade-off yang tajam terhadap kendala MVP. **Pertama**, BIGSERIAL memiliki jejak index yang jauh lebih kecil. UUID menempati 16 byte per primary key dibandingkan 8 byte untuk BIGSERIAL. Perbedaan 8 byte ini terlihat kecil, tetapi ketika dikalikan dengan setiap foreign key yang mereferensikan primary key tersebut, setiap secondary index yang menyertakan PK, dan setiap baris data, akumulasi overhead mencapai 30-50% peningkatan ukuran database. Pada VPS 2GB RAM dengan storage terbatas, setiap megabyte penting — terutama untuk query performance yang bergantung pada data fitting dalam memory buffer. **Kedua**, BIGSERIAL adalah default Laravel dan mengintegrasikan secara mulus dengan Eloquent ORM. Penggunaan UUID memerlukan custom casting, trait `HasUuids`, override `getKeyType()`, dan penyesuaian pada route model binding. Semua ini menambah kompleksitas kode yang tidak diperlukan untuk MVP. **Ketiga**, BIGSERIAL kompatibel dengan baik antara PostgreSQL dan MySQL, memberikan fleksibilitas jika di masa depan diperlukan migrasi database engine (lihat ADR-005). UUID generation mechanism berbeda antara PostgreSQL (`gen_random_uuid()`) dan MySQL (`UUID()` atau `binary(16)`), yang akan menjadi tantangan migrasi jika UUID digunakan sebagai PK. **Keempat**, untuk fase MVP dengan single-server deployment, UUID tidak memberikan keuntungan nyata. Keuntungan utama UUID — generation tanpa koordinasi antar node, tidak predictable, dan aman untuk distributed merge — baru relevan ketika sistem berjalan di multi-region dengan write replicas. MVP SIMT MTs tidak memiliki kebutuhan ini.

Konsekuensi

- **Harus mengimplementasikan strategi migrasi ketika scaling ke multi-region.** Ketika sistem perlu mendukung multi-database atau multi-region, primary key BIGSERIAL akan berisiko collision. Strategi migrasi yang direncanakan: (a) tabel baru di Phase 2 menggunakan UUID, (b) tabel existing ditambahkan kolom `uuid` sebagai alternate unique identifier, (c) foreign key tetap menggunakan BIGSERIAL untuk performa, (d) client-facing API secara bertahap bermigrasi ke UUID-based identifiers untuk menghindari enumeration attack.
- **ID bersifat sequential dan predictable.** Auto-increment ID memungkinkan enumeration — pengguna bisa menebak ID berikutnya. Mitigasi: gunakan UUID atau slug untuk semua client-facing identifier

(URL, API response), sedangkan BIGSERIAL hanya digunakan secara internal untuk join dan referential integrity.

- **Bulk insert lebih sederhana.** BIGSERIAL memungkinkan `INSERT` tanpa perlu generate ID terlebih dahulu, menyederhanakan seeder, factory, dan migration.
- `tenant_id` sebagai **UUID dapat diterima** karena tabel `tenants` berisi sedikit baris (puluhan hingga ratusan sekolah), sehingga overhead index minimal. UUID pada `tenant_id` memberikan keuntungan identifikasi unik global yang berguna untuk tenant resolution dan future multi-database partitioning.

Catatan Implementasi

- Semua migration menggunakan `$table->id()` (Laravel shorthand untuk BIGSERIAL)
- Tabel `tenants`: PK = `id` (BIGSERIAL), kolom tambahan `uuid` (UUID, unique) untuk identifikasi external
- Foreign key ke `tenants`: `$table->foreignId('tenant_id')` (BIGSERIAL, merujuk ke `tenants.id`)
- Jangan gunakan `$table->uuid()` sebagai primary key di tabel mana pun pada fase MVP
- Tambahkan index komposit pada semua foreign key yang sering di-query bersama `tenant_id`

Status: FINAL

ADR-003: RBAC Implementation — Spatie Laravel Permission with Teams

Konteks

Inkonsistensi implementasi RBAC terdapat antara **Dokumen 03** yang menggunakan pseudo-code Node.js/Express untuk implementasi role-based access control, dan **Dokumen 22 serta 27** yang menggunakan **Spatie Laravel Permission** package sebagai solusi RBAC. Dokumen 03 tampaknya ditulis pada fase awal ketika arsitektur backend masih belum ditentukan, sedangkan Dokumen 22 dan 27 merepresentasikan keputusan yang lebih matang setelah Laravel dipilih sebagai backend framework. Selain itu, terdapat pertanyaan apakah mode Teams pada Spatie perlu diaktifkan untuk mendukung multi-tenant role isolation.

Keputusan

Spatie Laravel Permission dengan `teams = true`, di mana `team_id` dipetakan ke `tenant_id`. Ini berarti setiap penugasan role/permission ke user bersifat per-tenant — seorang guru bisa memiliki role "Wali Kelas" di Madrasah A tetapi hanya "Guru Mata Pelajaran" di Madrasah B.

Rasional

Keputusan ini didasarkan pada empat alasan teknis yang kuat. **Pertama**, Spatie Laravel Permission adalah package Laravel-native yang paling banyak digunakan (15.000+ GitHub stars, actively maintained) dan terintegrasi langsung dengan Eloquent ORM, Blade directives, dan middleware Laravel. Tidak perlu menulis custom RBAC dari nol — ini menghemat ratusan jam pengembangan yang krusial untuk MVP 3 bulan. **Kedua**, mode Teams (`teams = true`) memungkinkan isolasi role antar tenant secara native. Tanpa Teams mode, seorang user yang ditugaskan role "Admin" di satu tenant akan memiliki role "Admin" di seluruh tenant — ini jelas tidak diinginkan dalam arsitektur multi-tenant SaaS. Dengan Teams mode, penugasan role bersifat scoped: `user->assignRole('admin', $tenantId)`. **Ketiga**, Spatie mengintegrasikan langsung dengan Blade directives `@can('edit-student')`, `@role('admin')`, dan `@hasanyrole('admin|teacher')` yang dapat digunakan langsung di template Blade tanpa logic tambahan. Untuk Next.js portal, permission check dilakukan via API middleware yang memeriksa `auth()->user()->hasPermissionTo('view-grades')`. **Keempat**, matriks permission yang komprehensif sudah dirancang dalam Dokumen 27, mencakup setiap aksi per role per modul. Spatie menyimpan permission sebagai string yang dapat didefinisikan sesuai konvensi ini

({action}-{resource})), misalnya create-student, view-grade, export-report.

Konsekuensi

- **HARUS mendaftarkan Tenancy sebagai singleton di AppServiceProvider.** Ini adalah Bug #1 yang diidentifikasi dalam Dokumen 47 — jika Tenancy class tidak di-register sebagai singleton, setiap resolusi container akan membuat instance baru yang menyebabkan tenant_id null atau salah. Implementasi: `app()->singleton(Tenancy::class, fn() => new Tenancy());`
- **HARUS memanggil setPermissionsTeamId() sebelum auth checks.** Pada setiap request, setelah tenant di-resolve oleh middleware, `auth()->user()->setPermissionsTeamId($tenantId)` harus dipanggil agar Spatie membaca permission yang benar untuk tenant tersebut. Ini harus dilakukan di tenant middleware atau di `AuthenticatedSessionController`.
- **Tipe data team_id di Spatie adalah unsignedBigInteger** yang harus kompatibel dengan tipe tenant_id di seluruh sistem. Karena ADR-002 menetapkan BIGSERIAL untuk tenant_id, kompatibilitas ini terjamin. JANGAN menggunakan UUID untuk tenant_id di database — gunakan BIGSERIAL yang cocok dengan team_id Spatie.
- **Permission seeding harus per-tenant.** Ketika tenant baru dibuat, seeder harus membuat role dan permission spesifik untuk tenant tersebut menggunakan `setPermissionsTeamId($newTenantId)` sebelum `createRole()` dan `givePermissionTo()`.
- **Tidak ada RBAC di sisi Next.js.** Portal orang tua hanya memiliki satu role (parent) dengan permission yang fixed. RBAC kompleks hanya ada di sisi admin (Laravel).

Catatan Implementasi

```
// config/permission.php
'teams' => true,

// TenancyServiceProvider.php atau AppServiceProvider.php
app()->singleton(\App\Services\Tenancy::class);

// TenantMiddleware.php
public function handle($request, Closure $next) {
    $tenant = $this->resolveTenant($request);
    app(\App\Services\Tenancy::class)->setTenant($tenant);
    if (auth()->check()) {
        auth()->user()->setPermissionsTeamId($tenant->id);
    }
    return $next($request);
}
```

Status: FINAL

ADR-004: Budget & Scope — Lean MVP Rp 5 Juta

Konteks

Terdapat dua skenario anggaran yang saling bertentangan. **Dokumen 01** mengusulkan anggaran **Rp 1.5 miliar per tahun** dengan tim lengkap (project manager, 2 backend developer, 2 frontend developer, UI/UX designer, QA, DevOps) untuk membangun sistem komprehensif dengan seluruh modul yang dirancang. **Dokumen 08** mengusulkan anggaran **Rp 5 juta total** untuk 3 bulan MVP yang dikerjakan oleh satu founder dengan bantuan AI (Cursor, Claude). Perbedaan ini bukan hanya masalah angka — ia menentukan scope fitur, timeline pengiriman, strategi hiring, dan definisi keberhasilan proyek secara keseluruhan.

Keputusan

Dokumen 08 (Rp 5 juta, 3 bulan MVP) adalah anggaran AKTIF yang berlaku sekarang. Dokumen 01 adalah anggaran aspirasional skala penuh untuk referensi masa depan ketika pendanaan telah tersedia. Semua keputusan pengembangan, prioritas fitur, dan alokasi sumber daya mengacu pada anggaran Dokumen 08.

Rasional

Keputusan ini diambil berdasarkan realitas operasional yang tidak dapat diabaikan. **Pertama**, kenyataan saat ini adalah satu founder tanpa pendanaan eksternal. Anggaran Rp 1.5 miliar mengasumsikan tim 8+ orang dengan gaji pasar Indonesia — ini tidak realistis tanpa funding round atau revenue yang sudah berjalan. Memaksakan rencana anggaran yang tidak sesuai realitas akan menghasilkan estimasi yang misleading dan keputusan arsitektural yang over-engineered. **Kedua**, MVP harus membuktikan commercial viability terlebih dahulu sebelum mencari pendanaan. Tujuan MVP bukan membangun sistem yang sempurna — tujuannya adalah memvalidasi bahwa Madrasah Tsanawiyah bersedia membayar untuk SaaS manajemen sekolah, bahwa fitur yang ditawarkan memang memecahkan masalah nyata, dan bahwa model bisnis subscription dapat berjalan. MVP yang terlalu besar dan terlalu lama berarti burn rate yang lebih tinggi tanpa validasi. **Ketiga**, bantuan AI (Cursor, Claude, ChatGPT) secara dramatis mengurangi kebutuhan developer headcount untuk coding tasks. Dengan AI pair programming, satu founder dapat menyelesaikan pekerjaan yang secara tradisional memerlukan 2-3 junior developers. Namun, AI tidak menggantikan kebutuhan untuk code review, architecture decisions, user testing, dan customer development — semua tetap menjadi tanggung jawab founder. **Keempat**, anggaran Rp 5 juta sudah mencakup VPS (Rp 300rb/bulan × 3 bulan = Rp 900rb), domain + SSL (Rp 200rb), tools subscription (Cursor Pro, dll ≈ Rp 1.5 juta), dan buffer untuk kebutuhan operasional. Ini tight tetapi feasible untuk solo founder yang menggunakan AI efficiently.

Konsekuensi

- **Semua pemotongan fitur sesuai Dokumen 37 berlaku.** Modul yang DITANGGUHKAN dari MVP: Tahfiz (hafalan Al-Quran tracking), Inklusi (pendidikan inklusif untuk ABK), Library (perpustakaan digital), E-Office (surat menyurat elektronik), dan Sarpras (pengelolaan sarana prasarana). Modul MVP terbatas pada: Manajemen Siswa, Manajemen Guru & Tenaga Pendidik, Absensi, Nilai/Rapor, Jadwal Pelajaran, Pembayaran SPP, dan Dashboard.
- **Dokumen 07 (roadmap 18 bulan) tetap valid sebagai visi jangka panjang** tetapi tidak didanai saat ini. Roadmap ini akan diaktifkan secara bertahap seiring dengan pertumbuhan revenue dan/atau pendanaan. Setiap fase pada roadmap memerlukan milestone revenue yang harus dicapai terlebih dahulu.
- **Tidak ada hiring dalam fase MVP.** Semua pekerjaan teknis dilakukan oleh founder dengan AI assistance. Setelah MVP ter-launch dan mendapatkan traction (minimal 5 paying schools), founder dapat mempertimbangkan hiring freelance part-time untuk customer support dan bug fixing.
- **UI/UX menggunakan template/wireframe, bukan custom design.** Budget tidak mencakup UI/UX designer. Admin panel menggunakan Blade template dengan Tailwind CSS + komponen dari open-source admin templates (seperti Sneat atau Volt). Portal orang tua menggunakan Next.js + shadcn/ui yang menyediakan komponen siap pakai.
- **Testing manual为主, automated testing minimal.** Unit test hanya untuk logic kritis (grading, attendance calculation, billing). Integration dan E2E testing ditangguhkan ke Phase 2.

Status: FINAL

ADR-005: Database Engine — MySQL 8 untuk MVP

Konteks

Inkonsistensi pilihan database engine terdapat antara **Dokumen 05, 06, 09** yang menspesifikasikan PostgreSQL 15 sebagai database engine utama, dan **Dokumen 38, 39** yang menspesifikasikan MySQL 8. Perbedaan ini berdampak pada DDL syntax (UUID generation, JSON column handling, full-text search),

extension requirements, ketersediaan hosting, dan kemudahan troubleshooting di komunitas developer Indonesia.

Keputusan

MySQL 8 untuk MVP. PostgreSQL dipertimbangkan untuk Phase 2 jika kebutuhan analitik (window functions lanjutan, CTE kompleks, JSONB advanced operations) menjadi kritis.

Rasional

Keputusan ini didasarkan pada lima pertimbangan praktis yang sangat relevan untuk konteks MVP. **Pertama**, MySQL lebih ringan dalam penggunaan RAM pada VPS 2GB. Dalam konfigurasi default, MySQL 8 menggunakan sekitar 400-600MB RAM, sedangkan PostgreSQL 15 dengan konfigurasi serupa menggunakan 600-900MB RAM karena `shared_buffers`, `work_mem`, dan `maintenance_work_mem` yang lebih agresif. Selisih 200-300MB ini sangat signifikan pada VPS yang juga menjalankan PHP-FPM, Nginx, Redis, dan (opsional) Node.js untuk Next.js. Setiap megabyte yang dihemat dari database engine dapat dialokasikan untuk PHP-FPM worker atau Redis cache, yang langsung berdampak pada concurrent user capacity. **Kedua**, ekosistem developer Indonesia jauh lebih familiar dengan MySQL. Mayoritas tutorial Laravel dalam Bahasa Indonesia, hosting shared lokal (Niagahoster, Hostinger Indonesia, IDCloudHost), dan komunitas PHP Indonesia semuanya menggunakan MySQL sebagai database default. Ini mengurangi friction untuk debugging, mencari solusi di forum, dan — jika di masa depan ada hiring — onboarding developer baru. **Ketiga**, setup replikasi MySQL untuk scaling di masa depan lebih sederhana dibandingkan PostgreSQL. MySQL master-slave replication terkenal straightforward dan well-documented, yang penting untuk sistem SaaS yang diharapkan tumbuh dari single-server ke read-replica architecture. **Keempat**, Laravel mendukung MySQL dan PostgreSQL secara merata — tidak ada fitur Eloquent yang eksklusif untuk satu engine. Schema Builder, Migrations, dan Eloquent ORM bekerja identis pada kedua engine. **Kelima**, schema MVP yang terdiri dari 12 tabel inti tidak menggunakan fitur spesifik PostgreSQL yang tidak dapat direplikasi di MySQL. JSON columns didukung oleh MySQL 8 (dengan JSON data type dan JSON functions), full-text search didukung via FULLTEXT index, dan window functions dasar didukung sejak MySQL 8.0.

Konsekuensi

- **Dokumen 05, 06, 09 harus diperbarui** untuk merefleksikan MySQL 8 sebagai database engine. Semua DDL syntax yang menggunakan PostgreSQL-specific features harus dikonversi: `gen_random_uuid()` menjadi `UUID()` atau `binary(16)`, JSONB menjadi JSON, `tsvector/tsquery` menjadi FULLTEXT INDEX, SERIAL menjadi INT AUTO_INCREMENT, dan `NOW()` tetap kompatibel.
- **UUID generation menggunakan `binary(16)` atau `UUID()` function.** Jika kolom UUID diperlukan (misalnya untuk `tenants.uuid`), gunakan `$table->uuid('uuid')` di Laravel migration yang menghasilkan CHAR(36) di MySQL — lebih sederhana daripada BINARY(16) meskipun sedikit kurang efisien storage-wise. Untuk MVP, CHAR(36) acceptable karena jumlah tenant sangat sedikit.
- **Full-text search menggunakan MySQL FULLTEXT** sebagai pengganti PostgreSQL `tsvector/tsquery`. MySQL FULLTEXT support pada InnoDB tersedia sejak versi 5.6+ dan memadai untuk kebutuhan pencarian siswa, guru, dan pembayaran dalam skala MVP. Untuk Phase 2, jika diperlukan search yang lebih canggih (fuzzy, autocomplete, faceted), pertimbangkan Meilisearch atau Elasticsearch sebagai dedicated search engine.
- **Tidak ada declarative partitioning native.** PostgreSQL memiliki `PARTITION BY RANGE/LIST/HASH` yang powerful. MySQL 8 juga mendukung partitioning, tetapi dengan batasan tertentu (misalnya foreign keys tidak didukung pada partitioned tables). Untuk MVP, partitioning tidak diperlukan — tabel absensi yang paling besar pun masih manageable tanpa partitioning untuk <50 sekolah.
- **Charset UTF8MB4 wajib.** Aplikasi SIMT MTs menangani nama dalam Bahasa Indonesia yang mungkin mengandung karakter khusus (emoji dalam catatan, aksara Arab untuk konteks keislaman). MySQL harus dikonfigurasi dengan `charset=utf8mb4` dan `collation=utf8mb4_unicode_ci`.

Catatan Implementasi

```
// config/database.php
'mysql' => [
    'driver' => 'mysql',
    'charset' => 'utf8mb4',
    'collation' => 'utf8mb4_unicode_ci',
    'engine' => 'InnoDB',
],

// .env
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
```

Status: FINAL

ADR-006: Caching & Queue — Redis Multi-Purpose

Konteks

Terdapat perbedaan pendekatan untuk mekanisme caching dan queue antara **Dokumen 09** yang menyarankan **file-based caching** untuk MVP (dengan alasan kesederhanaan dan tidak adanya dependency tambahan), dan **Dokumen 39** yang menspesifikasikan **Redis** untuk tiga fungsi sekaligus: cache, session storage, dan queue backend. Perbedaan ini signifikan karena menentukan apakah sistem dapat menangani concurrent access, background job processing, dan future horizontal scaling.

Keputusan

Redis untuk cache + session + queue sebagaimana didefinisikan dalam Dokumen 39. File-based caching hanya sebagai fallback jika Redis mengalami kegagalan, bukan sebagai default.

Rasional

Keputusan ini didasarkan pada empat analisis teknis yang menunjukkan bahwa Redis bukan luxury melainkan necessity, bahkan pada VPS 2GB. **Pertama**, queue adalah **mandatory** untuk pengiriman notifikasi WhatsApp. SIMT MTs mengirimkan notifikasi WA untuk berbagai event: konfirmasi pembayaran SPP, laporan absensi harian, pengumuman sekolah, dan reminder. Semua pengiriman WA harus melewati rate-limiting (untuk mematuhi batasan API provider seperti Fonnte atau Wablas), jitter (untuk menghindari deteksi spam), dan retry mechanism (untuk menangani kegagalan sementara). Tanpa queue system, pengiriman WA synchronously akan memblokir HTTP request dan menyebabkan timeout — ini bukan opsi. Redis menyediakan queue backend yang andal dengan Laravel's `Queue::push()` dan Job classes. **Kedua**, session storage di Redis memungkinkan future load balancing. Jika di kemudian hari VPS perlu di-scale dengan menambah server dan load balancer, session yang tersimpan di Redis terpusat dapat diakses oleh semua worker. File-based session terikat pada single server dan akan menjadi bottleneck scaling. Meskipun MVP berjalan di single VPS, menggunakan Redis dari awal menghindari migrasi session storage yang berisiko di kemudian hari. **Ketiga**, file-based caching menciptakan I/O bottleneck di bawah concurrent access. Ketika 50 guru mengakses dashboard secara bersamaan di pagi hari untuk mengisi absensi, file-based cache akan menghasilkan puluhan file read/write per detik yang membebani disk I/O. Redis yang beroperasi sepenuhnya di-memory dapat menangani ribuan operasi per detik tanpa I/O bottleneck. **Keempat**, Redis sangat efisien untuk multi-purpose pada VPS 2GB. Satu instance Redis dengan maxmemory 256mb sudah cukup untuk cache data, session storage, dan queue processing secara bersamaan. Dibandingkan dengan menjalankan tiga sistem terpisah (file cache + file session + database queue), Redis mengkonsumsi resource yang lebih terkontrol dan predictable.

Konsekuensi

- **Redis HARUS terinstall dan terkonfigurasi di VPS-1.** Instalasi Redis pada Ubuntu/Debian: `apt install redis-server`. Konfigurasi `redis.conf` harus mengatur `maxmemory 256mb` dan `maxmemory-policy allkeys-lru` untuk mencegah Redis mengkonsumsi seluruh RAM VPS. Tanpa memory limit, Redis dapat menyebabkan OOM (Out of Memory) pada VPS 2GB yang juga menjalankan MySQL, PHP-FPM, dan Nginx.
- **Supervisor memonitor proses Redis.** Konfigurasi Supervisor harus mencakup monitoring Redis process dan restart otomatis jika crash. Selain itu, Supervisor juga menjalankan Laravel queue worker: `php artisan queue:work redis --tries=3 --max-time=3600`.
- **Memory limit 256MB untuk Redis.** Dengan alokasi 256MB, Redis memiliki ruang yang cukup untuk: ~50MB cache (configuration, permission matrices, frequently-queried student/teacher lists), ~50MB sessions (supporting ~500 concurrent sessions), dan ~150MB queue (supporting backlogged WA notification jobs). Jika antrian WA membesar, LRU eviction akan membersihkan cache entries yang kurang sering diakses terlebih dahulu.
- **Fallback strategy jika Redis down.** Laravel mendukung multiple cache/session/queue stores. Konfigurasi: `primary = Redis`, `fallback = file/database`. Monitor Redis uptime dan kirim alert jika down. Namun, jika Redis down, queue processing WA akan berhenti — ini harus direspon segera.
- **WA notification dispatch pattern.** Setiap WA notification di-dispatch ke queue dengan: `dispatch(new SendWaNotification($message, $recipient))` → Worker memproses dengan rate-limiting (1 message per 3 seconds per tenant) → Jitter random 1-5 seconds → Retry max 3 kali dengan exponential backoff → Log keberhasilan/kegagalan ke `notification_logs` table.

Catatan Implementasi

```
// config/cache.php
'default' => env('CACHE_DRIVER', 'redis'),

// config/session.php
'driver' => env('SESSION_DRIVER', 'redis'),

// config/queue.php
'default' => env('QUEUE_CONNECTION', 'redis'),

// .env
REDIS_HOST=127.0.0.1
REDIS_PASSWORD=null
REDIS_PORT=6379
CACHE_DRIVER=redis
SESSION_DRIVER=redis
QUEUE_CONNECTION=redis

# /etc/redis/redis.conf
maxmemory 256mb
maxmemory-policy allkeys-lru
```

Status: FINAL

ADR-007: Tenant Isolation — Single-Database with Global Scope

Konteks

Meskipun semua dokumen secara konsisten menggunakan pendekatan **single-database multi-tenant**, detail implementasi bervariasi secara signifikan. Beberapa dokumen mengasumsikan middleware-only filtering, yang lain mengasumsikan global scope pada Eloquent model, dan ada yang menyarankan trait-based approach. Selain itu, terdapat pertanyaan tentang penanganan user yang beroperasi lintas tenant (super_admin, founder) dan apakah tabel `users` harus tunduk pada tenant isolation. Bug #1 dari Dokumen 47 (Tenancy singleton) juga harus ditangani dalam konteks ini karena langsung berdampak pada keandalan tenant resolution.

Keputusan

Single-Database Multi-Tenant dengan BelongsToTenant Global Scope sebagaimana didefinisikan dalam Dokumen 21, 22, 23, dan 39. **SEMUA tabel domain WAJIB memiliki kolom tenant_id dengan composite index.** Tabel users **DIKECUALIKAN** dari global scope — user lintas tenant didukung (super_admin, founder mengakses data di seluruh sekolah).

Rasional

Keputusan ini didasarkan pada lima pertimbangan arsitektural yang fundamental untuk operasional sistem. **Pertama**, VPS 2GB tidak dapat mendukung database-per-tenant. Pendekatan database-per-tenant memerlukan koneksi database terpisah per tenant, yang berarti setiap koneksi mengkonsumsi RAM MySQL yang signifikan (~10-20MB per connection pool). Dengan 50 tenant, ini berarti 500MB-1GB RAM hanya untuk MySQL connection pools — tidak tersisa untuk PHP-FPM, Redis, dan Nginx. Single-database dengan global scope memungkinkan satu koneksi database yang efisien melayani seluruh tenant. **Kedua**, global scope memberikan filtering yang transparan dan otomatis. Setiap query Eloquent yang menyertakan trait BelongsToTenant secara otomatis menambahkan WHERE tenant_id = ? — developer tidak perlu mengingat untuk menambahkan filter manual di setiap query. Ini mengurangi risiko data leakage antar tenant yang merupakan vulnerability paling kritis dalam aplikasi multi-tenant. Satu query yang lupa di-scope dapat mengekspos data sekolah lain — global scope mencegah ini secara systematic. **Ketiga**, pengecualian tabel users dari global scope diperlukan untuk mendukung cross-tenant user. Founder dan super_admin harus dapat mengakses data di seluruh tenant tanpa perlu memiliki akun terpisah per sekolah. Tanpa pengecualian ini, super_admin hanya bisa melihat data satu sekolah pada satu waktu, yang tidak memadai untuk platform management. **Keempat**, pendekatan ini kompatibel dengan Spatie Laravel Permission Teams mode (ADR-003). tenant_id pada global scope sejalan dengan team_id pada Spatie — keduanya menggunakan nilai yang sama untuk men-scope data dan permission. **Kelima**, composite index pada tenant_id + primary key memastikan performa query yang optimal. Tanpa composite index, query yang memfilter berdasarkan tenant_id harus melakukan full table scan atau menggunakan index yang kurang efisien. Composite index (tenant_id, id) memungkinkan database langsung menuju ke subset data milik tenant tanpa scanning seluruh tabel.

Konsekuensi

- **KRITIS: Class Tenancy HARUS di-register sebagai singleton di AppServiceProvider.** Ini adalah Bug #1 dari Dokumen 47. Jika Tenancy class tidak di-register sebagai singleton, setiap kali container melakukan resolusi, instance baru dibuat yang menyebabkan tenant_id bernilai null atau salah. Ini berarti global scope akan menambahkan WHERE tenant_id IS NULL yang mengembalikan result kosong — seluruh aplikasi akan tampak "kehilangan data". Implementasi yang benar:
`app()->singleton(\App\Services\Tenancy::class);`
- **Setiap model baru WAJIB menggunakan trait BelongsToTenant.** Tidak ada pengecualian kecuali tabel users. Developer harus menambahkan trait ini pada setiap Eloquent model baru yang dibuat. Code review checklist harus mencakup pemeriksaan ini. Tanpa trait, model akan mengakses data seluruh tenant — ini adalah security vulnerability.
- **Validasi unique HARUS di-scope ke tenant_id.** Ketika memvalidasi keunikan (misalnya NIS siswa harus unik), rule unique:students,nis saja tidak cukup karena NIS mungkin sama di sekolah berbeda. Gunakan: `Rule::unique('students', 'nis')->where('tenant_id', $tenantId)` atau implementasikan custom validation rule yang secara otomatis menambahkan scope tenant.
- **Route model binding HARUS menerapkan setelah tenant middleware.** Jika route model binding di-resolve sebelum tenant middleware menetapkan tenant_id, global scope tidak akan berfungsi dan data dari tenant lain dapat terbaca. Pastikan middleware priority mengikuti urutan:
`TenantMiddleware → auth → substitute (route model binding).`
- **Migration untuk setiap tabel baru harus menyertakan:**
`- Kolom $table->foreignId('tenant_id')->constrained()->cascadeOnDelete()`

- Composite index `$table->index(['tenant_id', 'id'])`
- Unique constraints harus menyertakan `tenant_id` sebagai bagian pertama composite unique

Catatan Implementasi

```
// app/Traits/BelongsToTenant.php
namespace App\Traits;

use App\Services\Tenancy;
use Illuminate\Database\Eloquent\Builder;

trait BelongsToTenant
{
    protected static function bootBelongsToTenant(): void
    {
        static::addGlobalScope('tenant', function (Builder $builder) {
            $tenancy = app(Tenancy::class);
            if ($tenantId = $tenancy->getTenantId()) {
                $builder->where('tenant_id', $tenantId);
            }
        });

        static::creating(function ($model) {
            $tenancy = app(Tenancy::class);
            if (!$model->tenant_id && $tenantId = $tenancy->getTenantId()) {
                $model->tenant_id = $tenantId;
            }
        });
    }
}

// AppServiceProvider.php — CRITICAL FIX
public function register(): void
{
    $this->app->singleton(\App\Services\Tenancy::class);
}
```

Status: FINAL — DO NOT REVISIT

Appendix: Document Update Checklist

Berikut adalah daftar lengkap 49 dokumen proyek beserta status update yang diperlukan berdasarkan keputusan ADR di atas. Setiap dokumen yang memerlukan perubahan ditandai dengan [UPDATE] beserta detail perubahan. Dokumen yang sudah konsisten dengan keputusan ADR ditandai dengan [OK].

No	Dokumen	Status	Detail Update
00	Project Overview	[OK]	Sudah konsisten dengan Blade + Next.js
01	Full-Scale Budget	[UPDATE]	Tambahkan catatan: "Anggaran aspirasional — lihat ADR-004. Anggaran aktif = Doc 08." Tandai sebagai NON-AKTIF untuk fase MVP
02	Business Model Canvas	[OK]	Konsisten dengan model SaaS subscription

No	Dokumen	Status	Detail Update
03	RBAC Design v1 (Node.js)	[UPDATE]	Tandai sebagai SUPERSEDED. Referensi ke ADR-003 (Spatie). Seluruh pseudo-code Node.js harus di-replace dengan Spatie implementation
04	Data Flow Diagram	[UPDATE]	Perbarui DFD untuk merefleksikan MySQL (bukan PostgreSQL) dan Redis sebagai cache/session/queue
05	Tech Stack v1 (PostgreSQL + Vue)	[UPDATE]	Kritikal: Ganti Vue.js 3 → Blade + Alpine.js; Ganti PostgreSQL → MySQL 8; Ganti Vite → Laravel Mix/Vite untuk Blade asset compilation
06	Database Schema (UUID PK)	[UPDATE]	Kritikal: Ganti semua <code>`gen_random_uuid()`</code> PK → <code>`BIGSERIAL`/\$table->id()`</code> ; Ganti PostgreSQL-specific syntax → MySQL syntax; Pastikan semua tabel memiliki <code>`tenant_id`</code> + composite index
07	18-Month Roadmap	[UPDATE]	Tambahkan catatan: "Roadmap jangka panjang — tidak didanai saat ini. Prioritas sesuai ADR-004." Tandai Phase 1 sesuai scope Doc 37
08	Lean MVP Budget (Rp 5M)	[OK]	Sudah konsisten — ini adalah anggaran AKTIF per ADR-004
09	Architecture v1 (PostgreSQL + Vue + File Cache)	[UPDATE]	Kritikal: Ganti PostgreSQL → MySQL 8; Ganti Vue.js → Blade + Alpine.js; Ganti file-based cache → Redis; Ganti UUID PK → BIGSERIAL
10	User Stories	[OK]	Konsisten — tidak ada dependency teknis
11	Wireframe Admin	[UPDATE]	Sesuaikan dengan Blade template approach (bukan SPA). Navigasi menggunakan full-page reload, bukan client-side routing
12	Wireframe Portal Ortu	[OK]	Konsisten dengan Next.js mobile-first
13	API Design	[UPDATE]	Perbarui endpoint admin untuk Blade (session-based auth, bukan API token). API endpoints hanya untuk Next.js portal
14	Authentication Design	[UPDATE]	Tambahkan <code>`setPermissionsTeamId()`</code> call setelah tenant resolution. Referensi ADR-003

No	Dokumen	Status	Detail Update
15	Multi-Tenant Architecture	[UPDATE]	Perbarui untuk merefleksikan BelongsToTenant global scope + singleton Tenancy. Referensi ADR-007
16	Deployment Strategy	[UPDATE]	Tambahkan Redis installation dan configuration. Perbarui database engine ke MySQL 8
17	Frontend Architecture (Vue)	[UPDATE]	SUPERSEDED — ganti seluruh konten dengan Blade + Alpine.js approach. Referensi ADR-001
18	Component Library (Vue)	[UPDATE]	SUPERSEDED — ganti dengan Blade anonymous components + Alpine.js directives. Referensi ADR-001
19	State Management (Vuex/Pinia)	[UPDATE]	SUPERSEDED — Alpine.js state management. Tidak ada Vuex/Pinia. Server-side state via Blade + Laravel
20	Testing Strategy	[OK]	Konsisten — minimal automated testing untuk MVP
21	Tenant Middleware Design	[OK]	Sudah konsisten dengan global scope approach. Pastikan singleton registration terdokumentasi
22	Permission Matrix	[OK]	Sudah konsisten dengan Spatie + Teams approach
23	Tenant Resolution	[OK]	Sudah konsisten — pastikan referensi ke ADR-007
24	Frontend Architecture v2 (Blade + Next.js)	[OK]	Sudah konsisten — ini adalah versi AKTIF per ADR-001
25	Admin Panel Design (Blade)	[OK]	Sudah konsisten dengan Blade approach
26	Parent Portal Design (Next.js)	[OK]	Sudah konsisten dengan Next.js approach
27	RBAC Design v2 (Spatie)	[UPDATE]	Tambahkan implementasi detail: singleton Tenancy, setPermissionsTeamId(), team_id type alignment. Referensi ADR-003
28	Notification System	[UPDATE]	Perbarui untuk menggunakan Redis queue (bukan synchronous). Tambahkan rate-limiting + jitter pattern untuk WA. Referensi ADR-006
29	WhatsApp Integration	[UPDATE]	Perbarui dispatch mechanism ke Redis queue. Tambahkan retry + exponential backoff pattern

No	Dokumen	Status	Detail Update
30	Payment Gateway	[OK]	Konsisten — tidak ada dependency pada keputusan ADR
31	Reporting Module	[OK]	Konsisten — pastikan query menggunakan tenant-scoped models
32	Attendance Module	[OK]	Konsisten — pastikan model menggunakan BelongsToTenant trait
33	Grading Module	[OK]	Konsisten — pastikan model menggunakan BelongsToTenant trait
34	Schedule Module	[OK]	Konsisten — pastikan model menggunakan BelongsToTenant trait
35	Student Management	[OK]	Konsisten — pastikan unique validation scoped ke tenant_id
36	Teacher Management	[OK]	Konsisten — pastikan model menggunakan BelongsToTenant trait
37	MVP Feature Cuts	[OK]	Sudah konsisten — ini adalah daftar scope AKTIF per ADR-004
38	Infrastructure v2 (MySQL)	[OK]	Sudah konsisten dengan MySQL 8 per ADR-005
39	Architecture v2 (MySQL + Redis)	[OK]	Sudah konsisten — ini adalah versi AKTIF. Redis untuk cache+session+queue per ADR-006, BIGSERIAL per ADR-002
40	VPS Configuration	[UPDATE]	Tambahkan Redis maxmemory 256mb config. Perbarui MySQL tuning untuk 2GB RAM. Tambahkan Supervisor config untuk Redis + queue worker
41	CI/CD Pipeline	[UPDATE]	Tambahkan step untuk Next.js build. Pastikan MySQL (bukan PostgreSQL) di CI environment
42	Monitoring & Logging	[UPDATE]	Tambahkan Redis monitoring (memory usage, connected clients, queue length). Perbarui alert thresholds
43	Security Checklist	[UPDATE]	Tambahkan tenant isolation testing (verifikasi data leakage). Tambahkan singleton Tenancy sebagai critical checkpoint
44	Performance Budget	[UPDATE]	Perbarui TTFB target berdasarkan Blade SSR (bukan SPA). Perbarui memory budget dengan Redis 256MB alokasi

No	Dokumen	Status	Detail Update
45	Onboarding Flow	[OK]	Konsisten — pastikan tenant creation triggers permission seeding
46	Subscription & Billing	[OK]	Konsisten — tidak ada dependency pada keputusan ADR
47	Bug & Issue Log	[UPDATE]	Bug #1 (Tenancy singleton) sekarang terdokumentasi di ADR-003 dan ADR-007. Tambahkan referensi silang
48	Sprint Planning	[UPDATE]	Perbarui estimasi berdasarkan Blade (bukan Vue). Sederhanakan task frontend karena tidak perlu SPA setup
49	Release Checklist	[UPDATE]	Tambahkan: Redis connectivity test, tenant isolation smoke test, Spatie Teams mode verification, MySQL charset verification

Ringkasan Update Prioritas

Prioritas	Dokumen	Alasan
P0 — Kritikal	05, 06, 09, 17, 18, 19	Mengandung teknologi yang secara eksplisit DITOLAK oleh ADR (Vue.js, PostgreSQL, UUID PK, file cache)
P1 — Tinggi	03, 13, 14, 15, 27, 28, 29	Memerlukan perubahan implementasi signifikan berdasarkan keputusan ADR
P2 — Sedang	01, 07, 16, 38, 40, 41, 42, 43, 44, 47, 48, 49	Memerlukan penyesuaian atau penambahan catatan
P3 — Rendah	04, 11, 39	Memerlukan minor updates atau sudah hampir konsisten

Dokumen ini diproduksi sebagai bagian dari proses rekayasa perangkat lunak SIMT MTs. Setiap keputusan bersifat FINAL dan mengikat selama fase MVP. Perubahan terhadap keputusan ini hanya dapat dilakukan melalui proses ADR revision formal yang memerlukan justifikasi teknis dan persetujuan founder.